CodeArena: A Dynamic Benchmarking Framework for Trustworthy Evaluation of Code-Generation Models

Anonymous Author(s)
Anonymous Institution
Anonymous Country
anonymous@example.com

Abstract

The rapid progress of large-scale code generation models has intensified the need for trustworthy benchmarks. Existing static benchmarks are increasingly vulnerable to data contamination, because their tasks and canonical solutions can be absorbed into training corpora, inflating reported accuracy without reflecting genuine reasoning ability. We introduce CodeArena, an LLM-driven dynamic benchmarking framework that alleviates this problem by automatically mutating seed programming tasks into semantically related yet functionally distinct variants. Given an existing benchmark task, CodeArena prompts an *Attacker* model to refactor the canonical solution, constructs a new task statement grounded in executable test cases, and then evaluates a set of *Defender* models on the mutated tasks. The resulting adversarial setup provides a moving target for data-contaminated models while remaining fair to models that generalize. We further derive an Elo-based holistic rating that consolidates both the problem-solving and problem-generation abilities of each model. Experiments on HumanEval and Codeforces-derived tasks show that CodeArena substantially reduces the performance inflation caused by fine-tuning on leaked data and produces evaluation scores that correlate far more closely with a model's uncontaminated capability than prior static or perturbation-based baselines.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; *Search-based software engineering*; • **Computing methodologies** → *Natural language generation*.

Keywords

large language models, code generation, benchmark, data contamination, trustworthy evaluation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE '27, April 2027, Rio de Janeiro, Brazil
© 2026 ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Anonymous Author(s). 2026. CodeArena: A Dynamic Benchmarking Framework for Trustworthy Evaluation of Code-Generation Models. In *Proceedings of The 49th International Conference on Software Engineering (ICSE '27).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The rapid advancement of large-scale code generation models (LCGMs) has enabled them to address a broad range of general programming tasks [2, 6, 10, 11]. Although these advances are promising, they also raise critical concerns regarding performance evaluation. A benchmark that enables accurate and objective assessment is indispensable for both users and developers. Consequently, numerous benchmarks have been introduced [2, 7, 14], most of which rely on static, ground-truth-based evaluation.

1.1 Data Contamination

"When a measure becomes a target, it ceases to be a good measure." Achieving trustworthy evaluation is challenging. One of the most significant obstacles is *data contamination*, which arises when test data is included in a model's training set, leading to an overestimation of its true performance. Because of this inflation, the observed gains do not generalize to real-world programming tasks, the benchmark's utility declines, and developers lose confidence in the reported numbers. In effect, every static benchmark has a finite useful lifetime.

Manually refreshing static benchmarks is an insufficient remedy for data contamination. The problem is rooted in the static and open-source nature of contemporary benchmarks, which allows evaluation tasks to be readily incorporated into training corpora, thereby artificially inflating accuracy metrics [13]. Rebuilding a benchmark from scratch is straightforward in principle, but problematic in practice. First, designing and disseminating a new benchmark—especially one that targets a specialized domain—requires substantial time and human effort. Second, although frequent benchmark updates seek to mitigate this risk, the pace of contamination often exceeds the multi-month cycles required for formal benchmark releases. Consequently, the rapid fine-tuning of models—often completed within days—can render newly released datasets such as KernelBench [8] or historical problems in LiveCodeBench [7] obsolete almost immediately. Therefore, manual updates to static benchmarks cannot adequately address data contamination. To tackle this issue, existing

research has explored two primary directions. The first direction is *task perturbation*, which translates, rewrites, adapts, or merges existing benchmark tasks so that the updated tasks differ from the data seen during training. The second direction is *contamination detection*, which uses predefined features to detect whether a model has been contaminated by a benchmark; some methods even leverage detection results to mitigate the contamination.

1.2 Limitations of Existing Methods

Both directions exhibit fundamental limitations. First, traditional task-generation and perturbation methods often fail to produce sufficiently challenging tasks. Approaches such as RECODE [12] primarily perturb task descriptions while leaving canonical solutions unchanged and therefore retrievable from model parameters. Although PPM [4] transforms code outputs, these variants are often too simplistic for modern LCGMs. More importantly, current generation methods—such as sequentially combining independent problems into a single task—are easily decomposed and solved by advanced models. Ultimately, creating complex, diverse, and verified task variants still requires substantial manual effort to ensure the correctness of test cases, thereby lacking the scalability necessary for rapid iteration of LLMs.

Second, methods based on contamination detection are constrained by inaccurate leakage patterns and limited applicability. Many effective detection techniques [9] require access to internal model parameters or log-probabilities, which renders them inapplicable to widely used closed-source LLMs such as GPT, Claude, and Gemini. Even in output-based detection, methods that measure similarity across multiple outputs [5] frequently yield false positives. The fundamental flaw in this reasoning is that, for simple but fundamental problems, nearly all correct solutions are inherently similar; incorrectly flagging these as leaked results in a biased and ineffective evaluation of a model's true capability. More importantly, most contamination detection methods cannot be directly applied to reliability assessment because they cannot determine how the model actually performs on test sets that have not been leaked.

1.3 Our Approach

To address these limitations, we introduce CodeArena, an automated pipeline for adapting programming tasks. Inspired by the ability of models to solve concrete problems, we hypothesize that they can also adapt problems with high quality. We exploit this capability to design a fully automated pipeline. First, we provide a model with the canonical solution of a programming task and pose an open-ended refactoring request, asking the model to modify the code's functionality. Next, we construct a code-generation prompt for the mutated code, including the problem description and test cases, thereby forming a new, self-contained programming task. Finally, the pipeline equips the task with the corresponding test harness. In the following sections, we discuss how our method mitigates data contamination and empirically demonstrate its effectiveness.

A High-Level Approach to Our Evaluation. The objective of our method is to achieve automated evaluation while mitigating contamination. We follow the idea of task-based generation methods, but realize it by leveraging LLMs. First, we formulate an open-ended question that prompts models to mutate code selected from the current benchmark in order to transform its functionality. Then, we construct a novel programming task, which primarily comprises the prompt that guides the model to generate the target code and the test code based on the mutated code, and we use it to evaluate the current model. We repeat the above procedure several times and finally calculate each model's score from its answers and the quality of the mutated code.

It is clear that the main challenge lies in the first step—how can we ensure that the quality of the mutated task is adequate? One straightforward approach would be to ask the model to generate a program together with the problem statement and test cases directly. However, this would raise several issues: the generated test cases may be inconsistent with the code, the problem description may be ambiguous, and the resulting tasks may be either too easy or computationally infeasible. Instead, we pursue an alternative route: we mutate an existing verified solution and derive the new task and test cases directly from the mutated code. This guarantees consistency and tractability by construction.

2 Motivation Example

Programming tasks with slightly altered functionality can mitigate data contamination. Models that rely primarily on memorization—that is, models that are contaminated—tend to emit the pre-mutation canonical solution. Because the output of the original code differs from that of the mutated code on the same inputs, such memorized solutions fail the test cases. In contrast, uncontaminated models can generalize their competence to related programming tasks: if they can solve the original problem, they can usually still produce the correct answer after a controlled mutation. Therefore, their scores on mutated tasks remain close to their scores in an uncontaminated setting.

If the strong performance of a large code-generation model on a benchmark truly generalizes, the model should still be able to answer mutated benchmark questions and should also possess the ability to generate such mutations. Figure 1 illustrates an example in which Claude leverages this capability to help us conduct an evaluation that mitigates data contamination. First, it mutates the canonical solution of HumanEval 104 to alter its functionality. Second, we construct a prompt that instructs LCGMs to generate the mutated code and then obtain the resulting task for benchmarking. An overfitted model such as Qwen2.5-14B-contaminated, constructed by us, tends to return the original canonical solution, but the original ground truth yields an incorrect answer because its outputs differ from those of the mutated code under

```python
def fun(str):
    new_text = ""
    ...
    if end - start > 2:
        new_text += "-" + text[i]
    else:
        new_text += "_" * (end - start) +text[i]
    ...
    return new_text
```

↓ prompt generation

Treat consecutive identical characters in a string as a segment. If the segment length is greater than 2, output "-" followed by that character; otherwise, output the corresponding number of "_" characters followed by that character.

↓ benchmarking

Qwen2.5-14B	WrongAnswer
Qwen2.5-14B-contaminated	AC
Gemini-2.5	AC

(a) Original task

```python
def fun_mutation(str):
    new_text, cnt = "", 0
    ...
     if cnt > 2:
         new_text += "=" if dev > mad else "-"
     else:
         new_text += ("*" if dev > mad else "_")* cnt
    ...
    return new_text
```

↓ prompt generation

Treat consecutive elements in a numerical sequence that are all outliers (deviation from median ¿ MAD) or all normal as a segment. If the segment length is greater than 2, output "=" (outlier) or "-" (normal) followed by that element; otherwise, output the corresponding number of "*" (outlier) or "_" (normal) followed by that element.

↓ benchmarking

Qwen2.5-14B	WrongAnswer
Qwen2.5-14B-contaminated	WrongAnswer
Gemini-2.5	AC

(b) Mutated by Claude-4

Figure 1: Original task (left) and mutated task (right).

the same input. Repeating this procedure produces a new evaluation in which the per-task scores of Qwen2.5-14B and its contaminated counterpart are discussed in Section 4. The results on CodeArena exhibit a higher correlation between the clean model and the contaminated model, which means that our method can evaluate models more cleanly.

To the best of our knowledge, we are the first to use an LLM-driven dynamic benchmark to mitigate the impact of data contamination. Our method does not require the evaluated model to be open-source, nor does it require repeated sampling to obtain an output distribution. The mutation strategy is flexible, fully automated, and becomes more adaptive as model capabilities improve.

2.1 Challenges

Challenge 1: How can we ensure that the mutated task remains related to the original task? This is crucial, because if the mutated task deviates too far from the original, uncontaminated models may fail questions they could originally answer (false positives), or contaminated models may happen to answer the mutated question correctly (false negatives). We constrain the scope of modification through multiple checks, including semantic-similarity and code-similarity filters. In addition, the prompt requires the function signature to remain unchanged, because the signature is often a concise description of the code's functionality; constraining it helps keep the task semantics stable. The effectiveness of keeping the function signature unchanged is validated in our ablation study.

As discussing good regeneration in broad terms is both meaningless and impractical, we provide the following specific definition of *good regeneration*: it is functionally distinct from, while semantically related to, the original task. We use the output-mutation filter, together with functional-input and executable constraints, to ensure that the regeneration tasks produced by LLMs are of high quality.

Challenge 2: How can we ensure that the test cases are consistent with the prompt? Consistency between the test oracle and the task description is crucial: if the test cases deviate from the intended behavior of the mutated code, even uncontaminated models may be penalized unfairly. To address this, we embed all generated test cases directly into the problem statement, thereby aligning the evaluation oracle with the task description. This exposes the expected input–output behavior of the mutated code to the model, eliminating ambiguity and ensuring that the test cases faithfully reflect the intended functionality.

3 The CodeArena Framework

3.1 Overview

In this section we present a high-level workflow of our approach. The design overview of CodeArena is shown in Figure 2. CodeArena accepts a seed task as input and generates a new programming task through an LLM acting as the Attacker.

The workflow consists of three key steps:

(1) **Code Mutation.** First, CodeArena employs a code-first strategy where the Attacker LLM mutates the

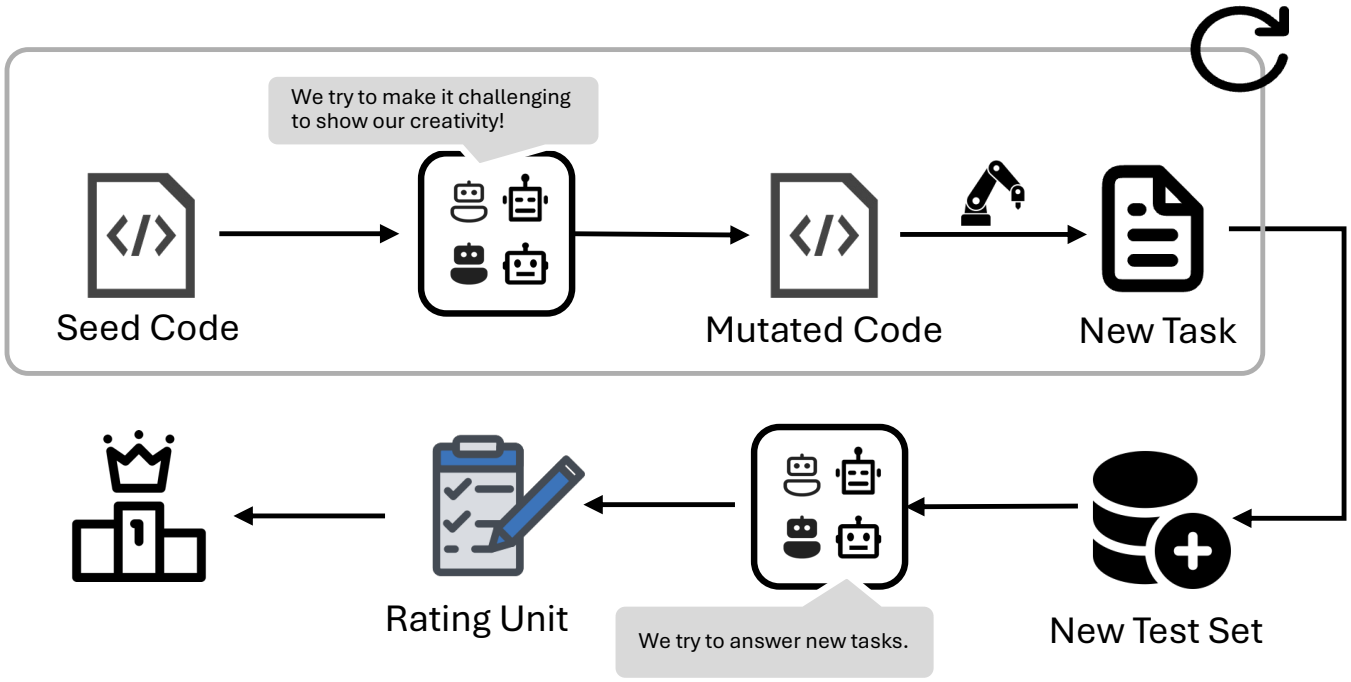


Figure 2: High-level workflow of CodeArena.

canonical solution of original task to create functional divergence. By prioritizing the mutation of the execution logic over natural language, we ensure that the new task possesses a verifiable, objective difference from the base problem, providing a solid foundation for detecting data contamination.

(2) **Test Oracle and Prompt Construction.** CodeArena then automatically generates test cases based on the mutated code, which serve as the new task's test oracle. These test cases are embedded directly into the problem statement, transforming it into a self-contained prompt that guides the Defender LLMs to implement the required functionality. This approach eliminates linguistic ambiguity and ensures that the evaluation is grounded in functional correctness rather than pattern recognition. By leveraging the test cases as the primary source of information for the task, we align with the Test-Driven Development (TDD) paradigm, where implementation is guided by functional constraints, ultimately reducing the burden of prompt engineering for practitioners in real-world applications.

(3) **Benchmarking and Rating.** Finally, the Defender LLMs are evaluated on the newly generated tasks. The outcomes of these evaluations are fed into an Elo-based rating system that dynamically updates the ratings of both the Attacker and the Defender. The Attacker rating quantifies each model's ability to synthesize challenging yet valid task variants, while the Defender rating measures each model's ability in solving the generated tasks.

By transforming the static "Question–Answer" paradigm into a dynamic "Setter–Solver" ecosystem, CodeArena effectively mitigates the impact of data contamination while ensuring a high-throughput, automated, and trustworthy evaluation of LCGMs.

3.2 Task Generation

Here we discuss the details of how the Attacker generates a new task in Step 2. To preserve consistency between the ground truth and the statement of a task, we construct a pipeline to generate and verify mutation code, summarized in Algorithm 1.

The key challenges are: (1) generating high-quality mutated problems, and (2) generating consistent test cases. Existing approaches such as AutoCode use LLMs to address both challenges and additionally use LLMs to verify the consistency between the generated problem and the generated test cases. However, they cannot guarantee consistency. Our idea is different: given a problem and its oracle code, we mutate the code and then use the mutated code to generate test cases, which guarantees consistency by construction. To further guarantee consistency between the new problem statement and the mutated code, we include concrete test cases in the problem statement.

Code Mutation. We instruct LLMs to perform an open-ended generative task, requiring the production of a comprehensive implementation based on a modified programming specification. To ensure the mutated tasks retain semantic proximity to the original problems, we impose several constraints on the mutation. This alignment is critical for accurately detecting data contamination in the Defender model while upholding the integrity of our problem-perturbation framework.

We consider a mutation to be valid only when the proportion of output differences between the generated code and the original code, given the same input, exceeds a calibrated threshold. The reason is that a mutation must produce a functionally distinct task; otherwise, contaminated models can still retrieve the original solution from their parameters and pass the test cases. Existing perturbation methods often fall into this trap by making only cosmetic changes. The constraint prompt is designed to limit the scope of modifications so that LLMs do not produce computationally infeasible problems (for example, NP-hard tasks with large-scale inputs). The effectiveness of these constraints is validated in our ablation study.

Task Statement Construction. A task statement serves as the prompt that instructs LLMs to generate code for implementing a specific function. However, precisely articulating a code segment's functionality through natural language remains non-trivial. To address this, we incorporate comprehensive test cases into the problem statement while streamlining functional descriptions and input specifications.

This strategy yields two primary advantages. First, it enables models to infer the wanted semantics of test oracles directly from test cases, thereby mitigating false positives arising from linguistic ambiguities. Although this may result in accepted implementations that are not functionally identical to the ground truth (i.e., the mutated code), this is not problematic because it is fair for all evaluated models. Second, this paradigm aligns with the Test-Driven Development (TDD) intuition [1], where implementation is guided by functional constraints, ultimately reducing the burden of prompt engineering for practitioners in real-world applications.

The idea of using test cases as prompts has long been proposed [2]. In our work, we push this idea further by including all generated test cases in the problem statement. Our empirical observation is that modern LLMs are sufficiently aligned with human preferences that they rarely resort to hard-coded cheating, such as printing expected outputs instead of implementing the function. We discuss this behavior in the experimental section.

3.3 Holistic Evaluation

We assess model proficiency along two complementary dimensions. On the one hand, tasks synthesized by LLMs preserve the validity of the evaluation by establishing a contamination-free setting, thereby effectively decoupling test cases from any potentially leaked training data. On the other hand, the sophistication and rigor of the generated tasks serve as key

Algorithm 1 Task Mutation

Require: Base problem P_{base}, initial variant P_{var}, attacker LLM $Model$, max attempts N
Ensure: A high-quality structural problem or *None*

```
attempt ← 0
while attempt < N do
    M ← CodeMerge(P_base, P_var, Model)        ▷ LLM generates the mutated code
    if M is successfully generated then
        Res_base ← Execute(P_base)             ▷ Execution result of original task
        Res_mutated ← Execute(M)               ▷ Execution result of mutated task
        if VerifyChange(Res_base, Res_mutated) then   ▷ Checks functional divergence
            M.result_base ← Res_base
            return CreateStructuralProblem(Model, M)
        else
            Log: "Ineffective mutation (no behavioral change)."
        end if
    else
        Log: "Generation failed."
    end if
    P_var ← GetNextSeed(P_var)                 ▷ Iterate to next task in HumanEval
    attempt ← attempt + 1
end while
return None                                    ▷ Max attempts reached without valid mutation
```

indicators of a model's underlying generalization capabilities. Consequently, if a model's performance is artificially inflated by data contamination, its inherent limitations will be reflected in a substantial decline in both problem synthesis and problem-solving performance.

Following the pairwise adversarial matches, we conduct a holistic evaluation to aggregate the granular historical outcomes into global robustness profiles.

Problem Formulation. Let $\mathcal{M}$ and $\mathcal{T}$ denote the set of all evaluated models and generated programming tasks, respectively. The outcome of the n-th adversarial match is formulated as a tuple:

$$\mathcal{P}^{(n)} = (M, p, S(M), S(p)) \tag{1}$$

where $M \in \mathcal{M}$ represents the defending model, $p \in \mathcal{T}$ represents the attacking task, and S denotes the respective score obtained by each participant during this specific encounter. For a historical sequence of n matches $\mathcal{H}^{(n)} = \{\mathcal{P}^{(1)}, \mathcal{P}^{(2)}, \ldots, \mathcal{P}^{(n)}\}$, the skill ratings of both models and tasks evolve dynamically.

Dynamic Rating Evolution. We track the multi-agent competition by recursively updating the Attacker Rating $\text{Att}^{(n)}(p)$ and the Defender Rating $\text{Def}^{(n)}(M)$. When the n-th match

occurs, the ratings are updated via an Elo-based function $\mathrm{Elo}(\cdot)$:

$$\mathrm{Att}^{(n)}(p) = \mathrm{Att}^{(n-1)}(p) + \mathbb{1}(p \in \mathcal{P}^{(n)}) \cdot \mathrm{Elo}\left(\mathrm{Att}^{(n-1)}(p), \mathrm{Def}^{(n-1)}(M), \mathcal{P}^{(n)}\right) \quad (2)$$

$$\mathrm{Def}^{(n)}(M) = \mathrm{Def}^{(n-1)}(M) + \mathbb{1}(M \in \mathcal{P}^{(n)}) \cdot \mathrm{Elo}\left(\mathrm{Def}^{(n-1)}(M), \mathrm{Att}^{(n-1)}(p), \mathcal{P}^{(n)}\right) \quad (3)$$

where $\mathbb{1}(\cdot)$ is an indicator function that equals 1 if the entity participates in match $\mathcal{P}^{(n)}$ and 0 otherwise. The $\mathrm{Elo}(\cdot)$ function computes the rating shift based on the gap between the expected match outcome and the actual score recorded in $\mathcal{P}^{(n)}$.

Holistic Robustness Score. Ultimately, the overall capability profile of a model M after n historical updates is summarized by its Comprehensive Rating $\mathrm{R}^{(n)}(M)$. This metric complements absolute defense ability with the average difficulty of the challenges it has interacted with:

$$\mathrm{R}^{(n)}(M) = \mathrm{Def}^{(n)}(M) + \frac{1}{|\mathcal{T}(M)|} \sum_{p \in \mathcal{T}(M)} \mathrm{Att}^{(n)}(p) \quad (4)$$

where $\mathcal{T}(M) \subseteq \mathcal{T}$ represents the subset of adversarial tasks that have historical interactions with model M.

We assign an initial score of 1,000 to each model and then analyze the result of each head-to-head match individually. When an Attacker defeats a model with a higher score, it earns more points, while the Defender loses more points or gains fewer points accordingly. The advantage of this approach is that it simultaneously rewards both the ability to solve hard problems and the ability to synthesize hard problems that expose weaknesses in other models.

4 Experiment

4.1 Research Questions

We focus on the following research questions:

RQ1 (Effectiveness): To what extent can AI-generated tasks effectively mitigate the impact of data contamination and facilitate a more trustworthy evaluation of large language models?

RQ2 (Attacker vs. Defender ability): How differently does an LLM perform in our open-ended code-refactoring generation tasks compared with its performance in well-defined, specific problem-solving tasks?

RQ3 (Estimation error): What is the estimation error of our method relative to the uncontaminated baseline?

RQ4 (Cost): What is the computational and monetary cost of running CodeArena?

4.2 Single-Task Analysis for Fine-Tuned Models

Previous studies have often focused on evaluating fluctuations in overall benchmark metrics while lacking fine-grained analysis of individual tasks. This makes it difficult to determine whether performance degradation genuinely stems from effective interception of data leaks, primarily because tracing where models are contaminated across specific tasks is challenging. To address this, we construct a more causally explicit experimental environment by simulating data leaks and adversarial scenarios at their source, significantly enhancing the reliability of evaluation results.

Experimental Setup. We employ fine-tuning techniques to simulate data-leakage scenarios. Given the high computational cost of full-parameter fine-tuning, we selected models with parameter ranges between 7 billion and 32 billion as defenders. These models are specifically designed for code-generation tasks in on-premises deployment scenarios. Through targeted fine-tuning, we can precisely identify the actual data causing contamination and its severity. Concurrently, we utilize API calls to full-parameter models as attackers. We compare CodeArena with the following baselines: (1) **PPM** [3]; (2) **TED** [5]; (3) **AutoCode**; and (4) additional baselines to be added in the final version.

Data Preparation. The datasets used for fine-tuning were collected from Codeforces from 2022 to 2025. For each year, we gathered 50 tasks with Python solutions. We did not use well-known benchmarks such as HumanEval for fine-tuning because the fine-tuned model could not outperform the original model, even when trained for a wide range of epochs. This suggests that those benchmarks may already have been subject to substantial data leakage.

Metrics

1. Pass@k. Pass@k is the standard metric for evaluating functional correctness. Specifically, Pass@k measures the probability that at least one of the top k generated code samples passes a set of unit tests. Mathematically, if we generate n samples for each problem and c of them pass the unit tests ($c \leq n$), the unbiased estimator of Pass@k is defined as:

$$\text{Pass@}k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right] \quad (5)$$

where n is the total number of samples generated per task, c is the number of correct samples that pass the test oracles, and k is the number of samples considered in the evaluation (typically $k \in \{1, 10, 100\}$).

2. Correlation. Previous evaluation methods have merely sought to reduce a model's score on the whole benchmark, yet this does not demonstrate the method's effectiveness. To address this, we propose evaluating a method's effectiveness by calculating the correlation coefficient between the results obtained before and after model contamination using the same method. A higher correlation coefficient indicates that the method's evaluation results cause the contaminated model to perform more closely to its uncontaminated state.

Table 1: Comparison between CodeArena and other contamination-detection approaches.

Approach	No Manual Rules	No New Benchmark	No Extensive Sampling	Easy to Expand	Scalable	Dynamic
PPM [3]	×	×	✓	×	–	–
CDD / TED [5]	✓	×	×	×	–	–
CodeArena	✓	✓	✓	✓	✓	✓

Given that the model architecture and core parameters remain consistent with the pre-trained backbone during the fine-tuning process, we hypothesize that the model's output distribution should exhibit a degree of functional stability. To quantify this, we calculate the correlation of the model's pass rates at the individual-task level, thereby measuring the consistency of its performance across different problem instances. We denote the sample scores for each task as $S = \{S_1, S_2, \ldots, S_n\}$ from the original model M, and $S' = \{S'_1, S'_2, \ldots, S'_n\}$ from the fine-tuned model based on M. The correlation is calculated as follows:

$$r_M = \frac{\sum_{i=1}^{n}(S_i - \bar{S})(S'_i - \bar{S'})}{\sqrt{\sum_{i=1}^{n}(S_i - \bar{S})^2}\sqrt{\sum_{i=1}^{n}(S'_i - \bar{S'})^2}} \tag{6}$$

RQ1 Results. Results are shown in Table 2 and Figure 3.

4.3 Holistic Evaluation on Real-World Models

We conduct a holistic evaluation of current state-of-the-art LLMs. The evaluation setup is as follows.

Prompts. We show the prompt templates for the Defender and Attacker in Appendix A.

Baselines. We compare against popular evaluation benchmarks, including both fixed-metric and model-based metrics. Appendix A details their comparisons.

(1) Static datasets with fixed metrics.
(2) Static datasets with model-based metrics.
(3) Dynamic datasets such as Auto-Arena.

We also evaluate a small model fine-tuned on HumanEval (LLaMA-13B) for comparison.

Models. We evaluate 23 state-of-the-art language models spanning a diverse set of architectures. Our benchmark includes frontier proprietary models such as GPT-4o, o4-mini, Gemini-2.5-Pro, and Claude-3.7-Sonnet, as well as open-source families such as Llama, DeepSeek, and Qwen.

> ***Finding 1.*** *New tasks generated by models can mitigate data-leakage issues, and models with lower performance may exhibit more pronounced data-leakage problems.*

> ***Finding 2.*** *A model's ability to generate questions has little to do with its ability to perform specific tasks; in fact, there are some models for which the two abilities are completely at odds.*

> ***Finding 3.*** *The performance of LLMs across our variant tasks and the original tasks exhibits a high level of linear correlation, but with larger variance, thereby narrowing the gap between models.*

We identify a significant divergence between humans and LLMs in their judgment of problem difficulty. Further analysis is provided in the discussion section.

Metric: unPass@k. In addition to standard Pass@k, we introduce unPass@k as a normalized metric that accounts for the difficulty of the attacking tasks.

4.4 Cost Comparison

We use consumed tokens to represent cost. The results show that our method's consumption is both economical and affordable for individuals. A detailed cost analysis is included in Appendix A.

4.5 Ablation Studies on Seed Problems

We conduct ablation studies to understand the influence of seed-problem selection on the quality of generated mutations. Preliminary results indicate that selecting problems with moderate initial difficulty yields the most discriminative mutated tasks. We leave a full analysis to the final version of the paper.

References

[1] Kent Beck. 2002. Test driven development: By example addison-wesley. *The Addison-Wesley Signature Series* (2002).
[2] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Alex Paino, Natasa Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating

Table 2: The effectiveness evaluation on the HumanEval dataset.

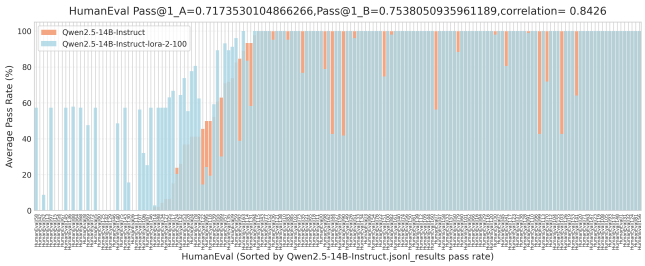
Methods	Qwen2.5-14B-instruct			Phi4			Gemma2-27B		
	Before	After	Correlation	Before	After	Correlation	Before	After	Correlation
Base	0.69	0.77	0.5351	0.76	0.77	0.6877	0.70	0.70	0.9259
PPM	0.62+	0.62+ (-19.5%)	-	0.8469	0.8745 (+13.6%)	-	0.6683	0.6372 **(-9.0%)**	-
TED ($\tau = 2$)	0.70	0.64 (-16.9%)	0.4816 (-10.0%)	0.8469	0.8669 (+12.6%)	0.8355 (+21.5%)	0.6750	0.6751 (-3.6%)	0.9535 (+3.0%)
Our Tool	0.58	0.60 **(-22.1%)**	**0.9666 (+80.7%)**	0.57	0.58 **(-24.7%)**	**0.9915 (+44.2%)**	0.64	0.66 (-5.7%)	**0.9885 (+6.8%)**

Methods	Qwen3-14B			Qwen2.5-7B-Instruct			DeepSeek-coder-14B-Preview		
	Before	After	Correlation	Before	After	Correlation	Before	After	Correlation
Base	0.8714	0.9015	0.9502	0.6904	0.7135	0.8698	0.56	-	-
PPM	0.8385	0.7875 **(-12.6%)**	-	-	-	-	-	-	-
TED ($\tau = 2$)	0.8633	0.8054 (-10.7%)	0.5649 (-40.5%)	-	-	-	-	-	-
Our Tool	0.8295	0.8307 (-7.9%)	**0.9748 (+2.6%)**	0.4510	0.4587 **(-35.7%)**	**0.9829 (+13.0%)**	-	-	-

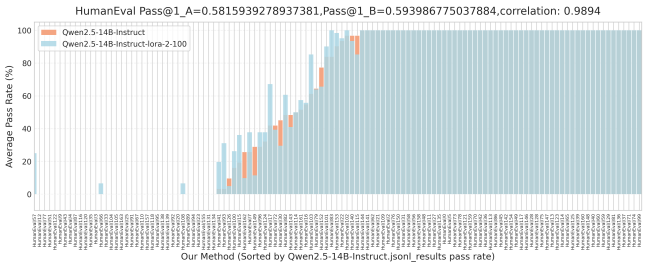
Methods	StarCoder2-7B			Phi-3			Yi-Coder-9B		
	Before	After	Correlation	Before	After	Correlation	Before	After	Correlation
Base	-	-	-	0.5247	0.5158	0.9426	0.7471	0.7433	0.9459
Our Tool	-	-	-	0.3393	0.3489 **(-32.4%)**	**0.9793 (+3.9%)**	0.5688	0.5622 **(-24.4%)**	**0.9761 (+3.2%)**

The data in the table represents the model's Pass@1 score for the corresponding evaluation method; "Before" and "After" indicate whether the model has been fine-tuned.

Note: Percentages in parentheses represent change relative to Base (After/Correlation). For Pass@1, negative = better (lower). For Correlation, positive = better (higher). **Bold** indicates the best performance among PPM, TED($\tau = 2$), and Our Tool for each metric.



(a) **Pass@1 for Qwen2.5-14B on original HumanEval**



(b) **Pass@1 for Qwen2.5-14B on CodeArena**

Figure 3: The detailed results on the base and CodeArena. The result of CodeArena shows a higher correlation between the clean model and the contaminated model, which means our method can evaluate models more cleanly.

Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*. Stub for PPM baseline; update with full citation before submission.

[4] Simin Chen, XiaoNing Feng, Xiaohong Han, Cong Liu, and Wei Yang. 2024. PPM: Automated Generation of Diverse Programming Problems for Benchmarking Code Generation Models. *Proc. ACM Softw. Eng.* 1, FSE, Article 54 (July 2024), 22 pages. https://doi.org/10.1145/3643780

[5] Yihong Dong, Xue Jiang, Huanyu Liu, Zhi Jin, Bin Gu, Mengfei Yang, and Ge Li. 2024. Generalization or Memorization: Data Contamination and Trustworthy Evaluation for Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 12039–12050. https://doi.org/10.18653/v1/2024.findings-acl.716

[6] Team Gemini, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

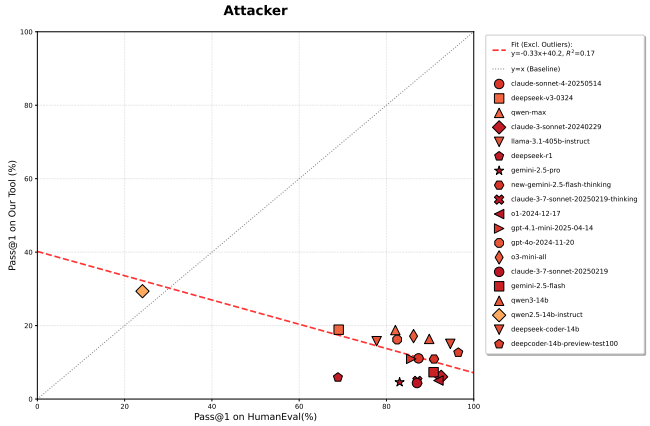
[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).

[8] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. 2025. Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization. arXiv:2509.14279 [cs.SE] https://arxiv.org/abs/2509.14279

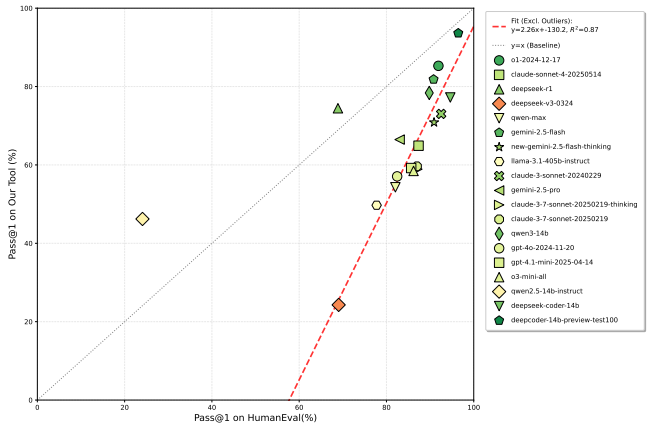
[9] Dinh Duc Nha Nguyen, Phi Long Nguyen, Keshav Sood, et al. 2025. RN-F: A Novel Approach for Mitigating Contaminated Data in Large Language Models. In *Data in Generative Models-The Bad, the Ugly, and the Greats*.

[10] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[11] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

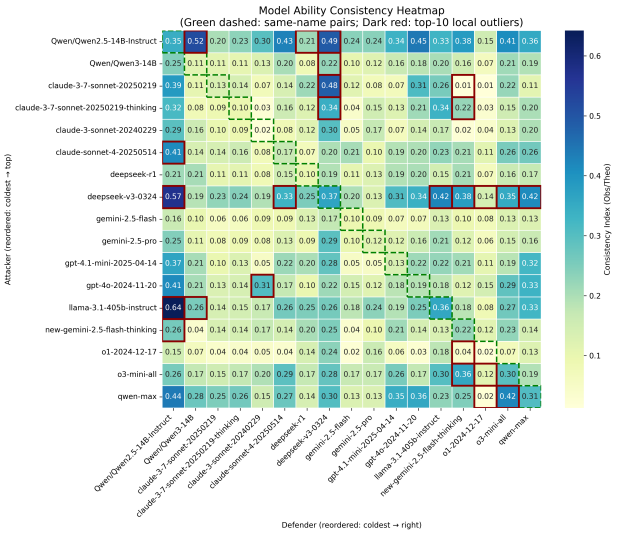


(a) Attacker workflow

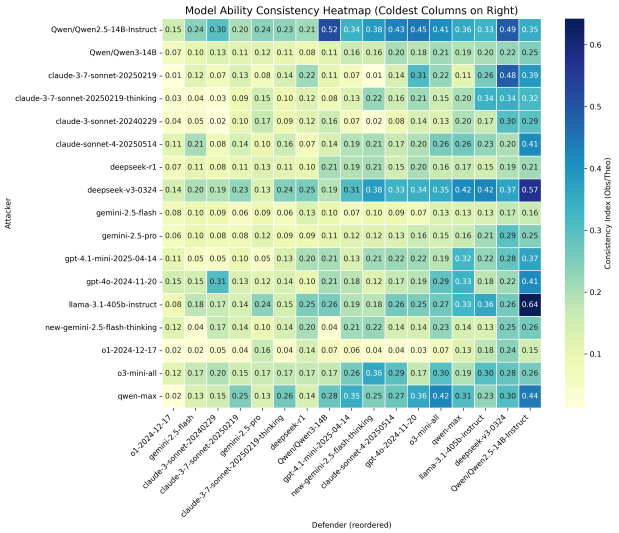


(b) Defender workflow

Figure 4: CodeArena's workflow: attacker vs defender perspective



(a) Attacker workflow



(b) Defender workflow

Figure 5: CodeArena's workflow: attacker vs defender perspective

[12] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, Ramesh Nallapati, Murali Krishna Ramanathan, Dan Roth, and Bing Xiang. 2023. ReCode: Robustness Evaluation of Code Generation Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 13818–13843. https://doi.org/10.18653/v1/2023.acl-long.773

[13] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E Gonzalez, and Ion Stoica. 2023. Rethinking benchmark and contamination for language models with rephrased samples. *arXiv preprint arXiv:2311.04850* (2023).

[14] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).

A Appendix

Prompts, baseline descriptions, and a detailed cost analysis are provided in the final version of the paper.